Rust - Strings

1. Introduction

Le type `String`, est le type de chaîne de caractères le plus utilisé en Rust.

Caractéristiques :

- Les chaînes sont encodées en UTF-8, ce qui signifie qu'un "caractère" peut occuper 1 à 4 octets, puisqu'une `String` peut contenir n'importe quel texte Unicode (lettres, chiffres, emojis...), et que l'UTF-8 est un encodage à largeur variable : chaque caractère (`char`) n'utilise pas le même nombre d'octets.

> **Encodage UTF-8**
>
> - Caractères ASCII (`a-z`, `0-9`, ponctuation basique) → 1 octet
> - Caractères accentués européens (`é`, `è`, `ç`) → 2 octets
> - Caractères moyen-orientaux et asiatiques → 3 octets
> - Emojis et caractères rares → 4 octets

- Rust expose cette complexité pour éviter les bugs liés aux caractères non ASCII tels que les emojis, les alphabets non latins, etc. Ce n'est pas une option, c'est une garantie du langage (Pour les chaînes qui ne sont pas en UTF-8, voire OsString).
- Une `String` est modifiable et allouée sur le tas. La `String` est propriétaire du texte qu'elle contient. C'est comme un `Vec<char>` mais optimisé pour le texte.
- Le fait qu'elle soit allouée sur le tas signifie que la taille de la chaîne peut grandir ou réduire pendant l'exécution du programme, contrairement aux chaînes littérales qui sont gravées dans le code binaire.
- Une `String` est "étroitement liée" au type primitif `str`. En pratique, on utilise presque toujours `str` sous la forme d'une référence : `&str`.
- `&str` : C'est une vue empruntée, immuable sur une séquence (ou tranche) de texte UTF-8.

2. Créer une `String`

Méthode	Exemple	Description
`String::new()`	`let mut s = String::new();`	Crée une `String` vide (sans allocation de mémoire - cf. infra).
`String::with_capacity(c)` (`c` pour capacité)	`let mut s = String::with_capacity(7);`	Créer une `String` vide (avec allocation mémoire)
`to_string()`	`let s = "hello".to_string()`	Convertit un littéral (`&str`) en `String` .
`String::from()`	`let s = String::from("hello");`	Crée une `String` à partir d'un littéral. `hello` est maintenant une `String` que l'on peut modifier.

Création à partir d'un vecteur d'octets UTF-8 :

```
fn main() {
    // Ces octets sont la représentation UTF-8 de l'emoji "💖".
    let coeur_etincelant = vec![240, 159, 146, 150];
    match String::from_utf8(coeur_etincelant) {
        Ok(emoji) => println!("Succès : {emoji}"),      // 💖
        Err(erreur) => println!("Échec : {}", erreur), // Échec :
invalid utf-8 sequence of...
    }
    // 'String::from_utf8' peut échouer si les octets ne sont pas du
UTF-8 valide.
}
```

3. Mettre à jour une `String`

Il faut pour cela que la `String` soit mutable (`let mut s = ...`).

Méthode	Exemple	Description
`push_str()`	`s.push_str(" world");`	Ajoute une tranche de chaîne (`&str`) à la `String` .
`push()`	`s.push('!');`	Ajoute une caractère (`char`) à la `String` .
`+` (concaténation)	`s1 + &s2`	Concatène deux `String` (consomme `s1`).
`format!`	`format!("{}{}"), s1, s2`	Concatène sans consommer les chaînes originales.

Exemple :

```
let mut s = String::from("hello");
s.push_str(" world");  // "hello world"
s.push('!');           // "hello world!"
```

4. Concaténation : `+` vs `format!`

- `+` :
  - consomme la première `String` (`s1 + &s2` rend `s1` inutilisable).
    - Moins lisible pour les concaténations multiples.

    ```
    let s3 = s1 + &s2;  // s1 est déplacée !
    ```
- `format!` :
  - Ne consomme aucune chaîne.
  - Plus lisible et flexible.

```
let s3 = format!("{}{}", s1, s2);  // s1 et s2 restent valides
```

5. Accès aux caractères

Pas d'indexation directe

- En UTF-8 , un caractère peut occuper plusieurs octets.
- Rust interdit `s[0]` pour éviter les erreurs (ex. : retourner un octet isolé au lieu d'un caractère valide).

La philosophie Rust : Plutôt que de faire des choix ambigus et potentiellement dangereux, le langage interdit l'indexation directe et force à choisir explicitement ce que nous voulons (octets, caractères, ou tranches).

Méthodes sûres

Méthode	Exemple	Description
`.chars()`	`s.chars().nth(0)`	Itère sur les caractères Unicode (`char`). Indice `0` : accès au premier caractère.
`.char_indices()`	`for (indice, caractere) in s.char_indices() {...`	Itérateur avec indices.
`.bytes()`	`s.bytes()`	Itère sur les octets bruts (`u8`).
`.graphemes()`	`s.graphemes(true)`	Itère sur les groupes de graphèmes (nécessite la crate `unicode-segmentation` - Cf crate unicode-segmentation).

Exemple :

```
use unicode_segmentation::UnicodeSegmentation;

fn main() {
    for g in "नमस्ते".graphemes(true) {
        println!("{g}");
    }  // "न", "म", "स्", "ते"
}
```

6. Découpage de chaînes (`Slicing`)

- Utiliser `&s[start..end]` pour extraire une tranche d'octets.
- Attention : Les plages doivent respecter les limites des caractères (sinon, Rust panique).

```
let s = "Здравствуйте";
let slice = &s[0..4];  // "Зд" (2 caractères × 2 octets)
// &s[0..1] => ❌ Panique : découpe au milieu d'un caractère !
```

7. Le trait `Deref` : conversions automatiques

`Deref` est un trait (interface) en Rust qui permet à un type d'agir comme s'il était un autre type. Quand un type implémente `Deref<Target = AutreType>`, Rust peut automatiquement convertir des références de ce type vers l'autre type.

`String` implémente `Deref<Target = str>`, ce qui permet deux choses :

1. Une `&String` se comporte comme une `&str` (conversion automatique)
2. `String` hérite alors de toutes les méthodes de `str` (`.len()`, `.trim()`, `.contains()`, etc.)

Code illustrant cela :

```
fn une_fonction_qui_attend_une_tranche(texte: &str) {
    println!("Longueur : {}", texte.len());
}

fn main() {
    let ma_string = String::from("Bonjour");

    // Grâce à DEREF : on peut passer &String à une fonction qui attend
&str
    une_fonction_qui_attend_une_tranche(&ma_string);

    // Grâce à DEREF : String a accès aux méthodes de str
    println!("{}", ma_string.contains("B")); // true

    // Solution idiomatique : &*
    une_fonction_qui_attend_une_tranche(&*ma_string);
    // Solution explicite : .as_str()
    une_fonction_qui_attend_une_tranche(ma_string.as_str());
}
```

Bonne pratique

Préférer `&str` comme paramètre de fonction. Cela accepte à la fois les `&String` (via `Deref`) et les littéraux `"texte"`.

8. Représentation mémoire d'une `String`

Une `String` est composée de trois éléments stockés sur la pile (stack) :

Composant	Rôle
Pointeur (`ptr`)	Adresse mémoire du *buffer* sur le tas (heap)
Longueur (`len`)	Nombre d'octets réellement utilisés
Capacité (`cap`)	Taille totale du *buffer* (en octets)

Règle : `len` ≤ `cap` (on ne peut pas dépasser le *buffer*). Le *buffer* contenant les données est toujours stocké sur le tas (heap).

```
Pile (stack)                    Tas (heap)
 ____________                    ____________________
| ptr ---------|---------------->| "Once upon a time" |
| len = 19     |                 | (19 octets)        |
| cap = 19     |                  ____________________
 ____________

Avec une capacité plus grande que la longueur :
 ____________                    ______________________
| ptr ---------|---------------->| "hello" [libre...]   |
| len = 5      |                 | (5 octets) (capacité)|
| cap = 16     |                  ______________________
 ____________
```

Inspection des composants :

```rust
let histoire = String::from("Il était une fois...");
println!("Longueur : {}", histoire.len());      // nb d'octets utilisés
println!("Capacité : {}", histoire.capacity()); // taille du buffer
```

De l'importance de la capacité : ajouter du texte ne réalloue que si la capacité est insuffisante, et il est préférable d'utiliser `String::with_capacity()` quand nous connaissons approximativement la taille finale (plus ou moins 20%) pour éviter les réallocations coûteuses.

```rust
// Sans pré-allocation
let mut s = String::new();
s.push_str("hello");  // capacité: 0 → 8
s.push_str("world");  // capacité: 8 → 16
```

```rust
// Avec pré-allocation (plus performant si on connaît la taille)
let mut s = String::with_capacity(25);
s.push_str("hello");  // capacité: 25 (inchangée)
s.push_str("world");  // capacité: 25 (inchangée)
```

9. Exemple pratique : suppression des accents avec mise en majuscule des caractères

```rust
fn main() {
    let with_accent = vec!["été", "enlève", "là", "âéîôù"];
    let without_accent: Vec<String> = with_accent
        .iter()
        .map(|&w| delete_accent_with_uppercase(&w))
        .collect();
    println!("{:?}", without_accent); // ["ETE", "ENLEVE", "LA",
"AEIOU"]
}

fn delete_accent_with_uppercase(word: &str) -> String {
    let word_without_accent: String = word
        .chars()
        .map(|c| {
            match c {
                // e accentués
                'é' | 'è' | 'ê' | 'ë' => 'E',
                // a accentués
                'à' | 'â' | 'ä' => 'A',
                // i accentués
                'î' | 'ï' | 'ì' => 'I',
                // o accentués
                'ô' | 'ö' => 'O',
                // u accentué
                'ù' => 'U',
                _ => c,
            }
        })
        .collect();
    return word_without_accent.to_uppercase();
}
```

10. Méthodes utiles de la bibliothèque standard

Méthode	Exemple	Description
`contains()`	`if s.contains("rust") { }`	Vérifie si une sous-chaîne est présente.
`replace()`	`s.replace("old", "new")`	Remplace toutes les occurrences d'une sous-chaîne.
`trim()`	`s.trim()`	Supprime les espaces en début/fin
`split()`	`s.split(",")`	Découpe la chaîne selon un délimiteur
`to_uppercase()`	`s.to_uppercase()`	Convertit en majuscules (respecte Unicode)
`to_lowercase()`	`s.to_lowercase()`	Convertit en minuscules (respecte Unicode)
`push_str()`	`s.push_str("monde");`	Ajoute une chaîne à la fin
`push()`	`s.push('!');`	Ajoute un seul caractère
`len()`	`let n = s.len();`	Retourne le nombre d'octets
`is_empty()`	`if s.is_empty() { }`	Vérifie si la chaîne est vide
`capacity()`	`let c = s.capacity();`	Retourne la capacité actuelle
`as_str()`	`let t: &str = s.as_str();`	Convertit en `&str`

Méthode	Exemple	Description
`clear()`	`s.clear();`	Vide la chaîne (garde la capacité)
`pop()`	`let c = s.pop();`	Supprime et retourne le dernier caractère
`remove()`	`let c = s.remove(2);`	Supprime un caractère à un index
`truncate()`	`s.truncate(5);`	Raccourcit à un nombre d'octets spécifiques et supprime ceux qui sont au-delà.
`split_whitespace()`	`let mots: Vec<&str> = texte.split_whitespace().collect();`	Découpe la chaîne à chaque espace, tabulation, retour à la ligne, etc. Retourne un itérateur sur les morceaux.
`lines()`	`for ligne in poeme.lines() { }`	Découpe la chaîne à chaque retour à la ligne (`\n`)

11. Ressources utiles

- Documentation officielle sur `String`
- Documentation sur `str`
- Crate `unicode-segmentation` (pour les groupes de graphèmes)
- The Rust Programming Language - `Strings`